

✓ 12 — Enhanced natural-movie stimulus features

Adds motion/temporal contrast, Fourier slope, orientation-energy summaries, and scene-change features. It reuses saved real-frame features when raw movie frames are unavailable.

✓ Publication safety boundary

This notebook is intended for publication analysis without overwriting long-running artifacts. By default, reruns write derived manuscript outputs to a versioned namespace such as `reports/tables/publication_upgrade_v2` and `reports/figures/publication_upgrade_v2`. Existing `data/`, `models/`, and canonical `reports/` artifacts should be treated as protected evidence unless an explicit regeneration plan is chosen.

```
from pathlib import Path
import os
import sys
import yaml
import numpy as np
import pandas as pd

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mov
").resolve()
SRC_DIR = PROJECT_ROOT / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
os.chdir(PROJECT_ROOT)

cfg_path = PROJECT_ROOT / "configs" / "publication_upgrade.yaml"
if not cfg_path.exists():
    raise FileNotFoundError(f"Could not find publication config: {cfg_path}")
with cfg_path.open("r", encoding="utf-8") as f:
    pub_cfg = yaml.safe_load(f)
cfg = pub_cfg

PUBLICATION_RUN_LABEL = os.environ.get("PUBLICATION_RUN_LABEL", "publication_u
ALLOW_CANONICAL_PUBLICATION_WRITE = os.environ.get("ALLOW_CANONICAL_PUBLICATION
if PUBLICATION_RUN_LABEL == "publication_upgrade" and not ALLOW_CANONICAL_PUBLI
    raise RuntimeError(
        "Refusing to write into the canonical publication_upgrade namespace. "
```

```

        "Use PUBLICATION_RUN_LABEL=publication_upgrade_v2 or set "
        "ALLOW_CANONICAL_PUBLICATION_WRITE=1 intentionally."
    )

from v1_manifold_publication.paths import PublicationPaths, ensure_publication
from v1_manifold_publication.readiness import build_readiness_report, format_

paths = PublicationPaths.from_config(cfg_path)
paths.publication_tables_dir = paths.root / "reports" / "tables" / PUBLICATION
paths.publication_figures_dir = paths.root / "reports" / "figures" / PUBLICATION
paths.versioned_processed_dir = paths.processed_dir / PUBLICATION_RUN_LABEL
ensure_publication_dirs(paths)

print("Using PROJECT_ROOT:", PROJECT_ROOT)
print("Publication run label:", PUBLICATION_RUN_LABEL)
print("Publication tables:", paths.publication_tables_dir)
print("Publication figures:", paths.publication_figures_dir)
print("Versioned processed feature dir:", paths.versioned_processed_dir)
print("Read-only readiness tier:", build_readiness_report(PROJECT_ROOT)["tier"]

```

```

Using PROJECT_ROOT: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication run label: publication_upgrade_v2
Publication tables: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication figures: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Versioned processed feature dir: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Read-only readiness tier: exploratory_or_scaffold

```

```

from v1_manifold_publication.stimulus_features import add_scene_change_score

WRITE_ENHANCED_FEATURES = os.environ.get("WRITE_PUBLICATION_OUTPUTS", "0") =
feature_files = (
    sorted(paths.processed_dir.glob("session_*frame_features.csv"))
    + sorted(paths.processed_dir.glob("session_*real_frame_features.csv"))
)
print("Candidate feature files:", len(feature_files))
print("WRITE_PUBLICATION_OUTPUTS:", WRITE_ENHANCED_FEATURES)

for feature_path in feature_files:
    df = pd.read_csv(feature_path)
    if "movie_frame" not in df.columns:
        continue
    if "temporal_absdiff_rms" in df.columns:
        df = add_scene_change_score(df)
    session_id = feature_path.name.split("_")[1]
    out = paths.versioned_processed_dir / f"session_{session_id}_publication
    if WRITE_ENHANCED_FEATURES:
        out.parent.mkdir(parents=True, exist_ok=True)
        df.to_csv(out, index=False)
        print("Saved", out, df.shape)
    else:

```

```

        print("Dry run; would save", out, df.shape)

if not WRITE_ENHANCED_FEATURES:
    print("Set WRITE_PUBLICATION_OUTPUTS=1 to write enhanced features into t

```

```

Candidate feature files: 4
WRITE_PUBLICATION_OUTPUTS: False
Dry run; would save C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Dry run; would save C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Dry run; would save C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Dry run; would save C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Set WRITE_PUBLICATION_OUTPUTS=1 to write enhanced features into the versioned

```

```

# Optional raw-frame extraction template.
# Replace `frames` with a numpy array shaped [n_frames, height, width] or [n
# from the Allen natural movie template if available.
from v1_manifold_publication.stimulus_features import extract_frame_feature_

RUN_RAW_FRAME_EXTRACTION = False
if RUN_RAW_FRAME_EXTRACTION:
    frames = np.load(paths.processed_dir / "natural_movie_frames.npy")
    enhanced = extract_frame_feature_table(frames, resize_shape=tuple(cfg["s
    enhanced = add_scene_change_score(enhanced)
    save_table(enhanced, paths.publication_tables_dir / "12_enhanced_movie_f
    display(enhanced.head())
else:
    print("Set RUN_RAW_FRAME_EXTRACTION=True after saving natural movie fram

```

```

Set RUN_RAW_FRAME_EXTRACTION=True after saving natural movie frames as data/p

```

✓ Extract enhanced movie features

This cell produces versioned publication result tables when the notebook is run in the full scientific environment.

```

# -----
# This cell extracts and SAVES enhanced stimulus features for all sessions
# listed in data/external/allen_v1_natural_movie_experiments.csv.
# -----

import os
from pathlib import Path

import numpy as np
import pandas as pd

```

```

from v1_manifold_publication.stimulus_features import (
    extract_frame_feature_table,
    add_scene_change_score,
    feature_hierarchy_columns,
)

# -----
# Settings
# -----

# IMPORTANT:
# This is now set to True, so the enhanced feature files will actually be saved
WRITE_ENHANCED_FEATURES = True

# Keep Gabor-like features because they are important for V1/natural-image analysis
INCLUDE_GABOR = True

# 0 means process all metadata rows.
MAX_FEATURE_SESSIONS = 0

# -----
# Locate metadata and output directories
# -----

metadata_path = paths.root / "data" / "external" / "allen_v1_natural_movie_events"

if not metadata_path.exists():
    raise FileNotFoundError(f"Missing metadata table: {metadata_path}")

experiments = pd.read_csv(metadata_path)

if MAX_FEATURE_SESSIONS > 0:
    experiments = experiments.head(MAX_FEATURE_SESSIONS)

# Use versioned processed directory if available; otherwise fall back to data directory
versioned_processed_dir = getattr(
    paths,
    "versioned_processed_dir",
    paths.root / "data" / "processed" / "publication_upgrade",
)

versioned_processed_dir.mkdir(parents=True, exist_ok=True)
paths.publication_tables_dir.mkdir(parents=True, exist_ok=True)

print("Metadata table:", metadata_path)
print("Number of sessions in metadata:", len(experiments))
print("Enhanced features will be saved to:", versioned_processed_dir)

# -----
# Load Allen Brain Observatory cache
#

```

```

# -----

feature_status = []
hierarchy_rows = []

try:
    from v1_manifold.data_access import get_boc, get_experiment_data, get_st
    from v1_manifold.config import load_config, get_paths

    base_cfg = load_config(paths.root / "configs" / "default.yaml")
    base_paths = get_paths(base_cfg)
    boc = get_boc(base_paths.allen_manifest)

    print("Allen Brain Observatory cache loaded successfully.")

except Exception as exc:
    boc = None
    feature_status.append({
        "stage": "allen_import_or_manifest",
        "status": "failed",
        "error": repr(exc),
    })
    print("Failed to load Allen Brain Observatory cache.")
    print("Reason:", repr(exc))

# -----
# Extract and save enhanced feature tables
# -----

if boc is not None:
    stimulus_name = cfg.get("cohort", {}).get(
        "natural_movie_stimuli",
        ["natural_movie_one"],
    )[0]

    resize_shape = tuple(
        cfg.get("stimulus_features", {}).get("resize_shape", [96, 160])
    )

    gabor_frequencies = tuple(
        cfg.get("stimulus_features", {}).get(
            "gabor_frequencies",
            [0.04, 0.08, 0.16, 0.24],
        )
    )

    gabor_orientations_deg = tuple(
        cfg.get("stimulus_features", {}).get(
            "gabor_orientations_deg",
            [0, 22.5, 45, 67.5, 90, 112.5, 135, 157.5],
        )
    )

```

```

)

for _, row in experiments.iterrows():
    session_id = str(int(row.get("id", row.get("session_id"))))

    print("\nProcessing session:", session_id)

    try:
        data_set = get_experiment_data(boc, int(session_id))

        template = get_stimulus_template(
            data_set,
            stimulus_name,
        )

        if template is None:
            raise RuntimeError(
                f"Allen stimulus template was not available for {stimulu
            )

        features = extract_frame_feature_table(
            template,
            resize_shape=resize_shape,
            include_gabor=INCLUDE_GABOR,
            gabor_frequencies=gabor_frequencies,
            gabor_orientations_deg=gabor_orientations_deg,
        )

        features = add_scene_change_score(features)

        out = (
            versioned_processed_dir
            / f"session_{session_id}_publication_enhanced_frame_features
        )

        if WRITE_ENHANCED_FEATURES:
            features.to_csv(out, index=False)
            status = "saved"
        else:
            status = "dry_run"

        feature_status.append({
            "session_id": session_id,
            "status": status,
            "n_frames": int(len(features)),
            "n_features": int(len(features.columns)),
            "output": str(out),
        })

    for group, cols in feature_hierarchy_columns(features).items():
        hierarchy_rows.append(f

```

```

        hierarchy_rows.append({
            "session_id": session_id,
            "feature_group": group,
            "n_available_features": int(len(cols)),
            "features": ",".join(cols),
        })

    print(f"Saved: {out}")
    print("Shape:", features.shape)

except Exception as exc:
    feature_status.append({
        "session_id": session_id,
        "status": "failed",
        "error": repr(exc),
    })

    print("Failed for session:", session_id)
    print("Reason:", repr(exc))

# -----
# Save status and hierarchy tables
# -----

feature_status = pd.DataFrame(feature_status)
hierarchy_table = pd.DataFrame(hierarchy_rows)

save_table(
    feature_status,
    paths.publication_tables_dir / "12_enhanced_feature_extraction_status.csv"
)

save_table(
    hierarchy_table,
    paths.publication_tables_dir / "12_feature_hierarchy_availability.csv",
)

display(feature_status)
display(hierarchy_table)

# -----
# Verify saved enhanced feature files
# -----

enhanced_files = sorted(
    versioned_processed_dir.glob("session_*_publication_enhanced_frame_featu"
)

print("\nSaved enhanced feature files:", len(enhanced_files))

for f in enhanced_files:

```

```
print(f.name, "|", round(f.stat().st_size / 1024, 2), "KB")

if WRITE_ENHANCED_FEATURES and len(enhanced_files) == 0:
    raise RuntimeError(
        "WRITE_ENHANCED_FEATURES=True, but no enhanced feature files were sa
    )
```

Metadata table: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Number of sessions in metadata: 3
Enhanced features will be saved to: C:\Users\Peter\Documents\projects\NeuroAI
Allen Brain Observatory cache loaded successfully.

Processing session: 500855614
Saved: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
Shape: (900, 105)

Processing session: 500964514
Saved: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
Shape: (900, 105)

Processing session: 501271265
Saved: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
Shape: (900, 105)

session_id status n_frames n_features				
0	500855614	saved	900	105 \Users\Peter\Documents\projects\Neuro
1	500964514	saved	900	105 \Users\Peter\Documents\projects\Neuro
2	501271265	saved	900	105 \Users\Peter\Documents\projects\Neuro

session_id		feature_group	n_available_features
0	500855614	level1_luminance_contrast	6 luminance_mean,l
1	500855614	level2_spatial_orientation	10 spatial_frequency
2	500855614	level3_temporal_motion	9 temporal_absdiff_
3	500855614	level4_v1_like_gabor	64 gabor_energy_f0p0
4	500855614	level5_deep_model	0
5	500964514	level1_luminance_contrast	6 luminance_mean,l
6	500964514	level2_spatial_orientation	10 spatial_frequency
7	500964514	level3_temporal_motion	9 temporal_absdiff_
8	500964514	level4_v1_like_gabor	64 gabor_energy_f0p0
9	500964514	level5_deep_model	0
10	501271265	level1 luminance contrast	6 luminance mean,l

11	501271265	level2_spatial_orientation	10	spatial_frequency
12	501271265	level3_temporal_motion	9	temporal_absdiff_
13	501271265	level4_v1_like_gabor	64	gabor_energy_f0p0
14	501271265	level5_deep_model	0	

Saved enhanced feature files: 3

session 500855614 publication enhanced frame features.csv | 1774.93 KB

```
# -----
# Find enhanced feature files wherever notebook 12 saved them
# -----

processed_root = paths.root / "data" / "processed"

print("Processed root:", processed_root)
print("Processed root exists:", processed_root.exists())

# Search recursively inside data/processed
feature_files = sorted(
    processed_root.rglob("session_*_publication_enhanced_frame_features.csv")
)

tensor_files = sorted(
    paths.interim_dir.rglob("session_*_tensor.h5")
)

embedding_files = sorted(
    paths.processed_dir.rglob("session_*_embeddings.npz")
)

print("\nEnhanced feature files found:", len(feature_files))
for f in feature_files:
    print(" ", f)

print("\nTensor files found:", len(tensor_files))
for f in tensor_files:
    print(" ", f)

print("\nEmbedding files found:", len(embedding_files))
for f in embedding_files:
    print(" ", f)

# Compact session-level summary
def extract_session_id(path):
    name = path.name
    parts = name.split("_")
    for p in parts:
        if p.isdigit():
```

```

    if p.isnumeric():
        return p
    return None

feature_sessions = sorted({extract_session_id(f) for f in feature_files})
tensor_sessions = sorted({extract_session_id(f) for f in tensor_files})
embedding_sessions = sorted({extract_session_id(f) for f in embedding_files})

print("\nSessions with enhanced features:", feature_sessions)
print("Sessions with tensors:", tensor_sessions)
print("Sessions with embeddings:", embedding_sessions)

complete_neural_sessions = sorted(
    set(feature_sessions)
    & set(tensor_sessions)
    & set(embedding_sessions)
)

print("\nSessions ready for full neural analysis:", complete_neural_sessions)

```

```

Processed root: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Processed root exists: True

Enhanced feature files found: 4
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies

Tensor files found: 1
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies

Embedding files found: 1
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies

Sessions with enhanced features: ['500855614', '500964514', '501271265']
Sessions with tensors: ['500855614']
Sessions with embeddings: ['500855614']

Sessions ready for full neural analysis: ['500855614']

```

